

Revisão Eletrônica Digital

Prof. Roberto de Matos

roberto.matos@ifsc.edu.br



**INSTITUTO
FEDERAL**
Santa Catarina

Câmpus
São José

Objetivo

- Relembrar conceitos de eletrônica digital.
- Relembrar blocos de construção digital (combinacional e sequencial).



Sistemas de Numeração

- Base 10: Possibilidade de variar 10 algarismos diferentes (0, ..., 9) em cada posição.
- Da direita para a esquerda, os pesos de cada coluna são 1, 10, 100, 1000, ...
- $9742_{10} = 9 \times 10^3 + 7 \times 10^2 + 4 \times 10^1 + 2 \times 10^0$



- Base 2: Possibilidade de variar 2 algarismos diferentes (0, 1) em cada posição.
- Da direita para a esquerda, os pesos de cada coluna são 1, 2, 4, 8, 16, ...
- $10110_2 = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 22_{10}$
- Para representar a mesma quantidade de informações, precisamos usar mais posições (bits) em binário.



- Base 2: Possibilidade de variar 2 algarismos diferentes (0, 1) em cada posição.
- Da direita para a esquerda, os pesos de cada coluna são 1, 2, 4, 8, 16, ...
- $10110_2 = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 22_{10}$
- Para representar a mesma quantidade de informações, precisamos usar mais posições (bits) em binário.

Dicas:

- Um número binário de N bits pode representar 2^N possibilidades, ou seja, 0, 1, 2, 3, ..., $2^N - 1$.
- Para saber exatamente o número de bits (N) necessários para representar um determinado valor decimal (D) use a seguinte fórmula: $N = \lfloor \log_2(D) \rfloor + 1$



- Base 16: Possibilidade de variar 16 algarismos ($0, \dots, 9, A, \dots, F$) diferentes em cada posição.
- Da direita para a esquerda, os pesos de cada coluna são 1, 16, 256, 4096, 65.536, ...
- $2ED_{16} = 2 \times 16^2 + E \times 16^1 + D \times 16^0 = 749_{10}$, sendo $E = 14_{10}$ e $D = 13_{10}$



Hexadecimal

- Base 16: Possibilidade de variar 16 algarismos (0, ..., 9, A, ..., F) diferentes em cada posição.
- Da direita para a esquerda, os pesos de cada coluna são 1, 16, 256, 4096, 65.536, ...
- $2ED_{16} = 2 \times 16^2 + E \times 16^1 + D \times 16^0 = 749_{10}$, sendo $E = 14_{10}$ e $D = 13_{10}$

Dicas:

- Facilita a escrita de binários longos.
- 4ª potência da base 2 ($2^4 = 16$), ou seja, cada hexadecimal representa 4 bits.
- Colocar 0x na frente de um hexadecimal é uma notação muito utilizada (ex.: 0x2ED).



Representação de números negativos

- Esta é a opção adotada para representar números negativos em praticamente todos os computadores e outros sistemas digitais.
- A representação binária de um número negativo é obtida da seguinte forma:
 - 1 Converta a sua representação positiva (magnitude) para binário
 - 2 Inverta todos os bits (complemento de um)
 - 3 Some um (1)
- Exemplo: Converter -7_{10} considerando uma representação de 5 bits.

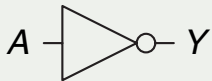


- Esta é a opção adotada para representar números negativos em praticamente todos os computadores e outros sistemas digitais.
- A representação binária de um número negativo é obtida da seguinte forma:
 - 1 Converta a sua representação positiva (magnitude) para binário
 - 2 Inverta todos os bits (complemento de um)
 - 3 Some um (1)
- Exemplo: Converter -7_{10} considerando uma representação de 5 bits.
 - 1 $+7_{10} = 00111_2$
 - 2 11000_2
 - 3 $11000_2 + 1 = 11001_2$
- Assim, $-7_{10} = 11001_2$



Portas lógicas

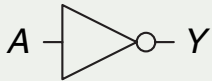
NOT



$$Y = \bar{A}$$

A	Y
0	1
1	0

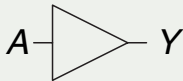
NOT



$$Y = \bar{A}$$

A	Y
0	1
1	0

BUF

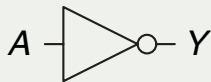


$$Y = A$$

A	Y
0	0
1	1



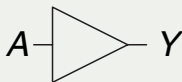
NOT



$$Y = \bar{A}$$

A	Y
0	1
1	0

BUF



$$Y = A$$

A	Y
0	0
1	1

AND



$$Y = AB$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1



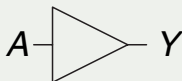
NOT



$$Y = \bar{A}$$

A	Y
0	1
1	0

BUF



$$Y = A$$

A	Y
0	0
1	1

AND



$$Y = AB$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR

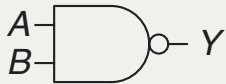


$$Y = A + B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1



NAND

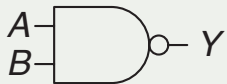


$$Y = \overline{AB}$$

<i>A</i>	<i>B</i>	<i>Y</i>
0	0	1
0	1	1
1	0	1
1	1	0



NAND



$$Y = \overline{AB}$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

NOR



$$Y = \overline{A+B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

NAND



$$Y = \overline{AB}$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

NOR



$$Y = \overline{A+B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

XOR



$$Y = A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0



NAND



$$Y = \overline{AB}$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

NOR



$$Y = \overline{A+B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

XOR



$$Y = A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

XNOR

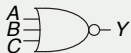


$$Y = \overline{A \oplus B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1



NOR3



$$Y = \overline{A + B + C}$$

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0



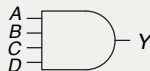
NOR3



$$Y = \overline{A + B + C}$$

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

AND4



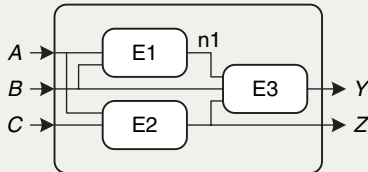
$$Y = ABCD$$

A	C	B	D	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

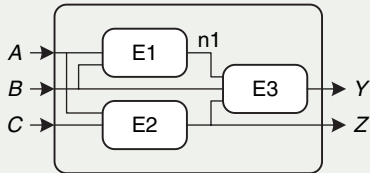


Lógica Combinacional

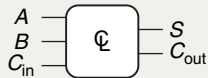
- Saída tem relação só com entradas:



- Saída tem relação só com entradas:



- Somador Completo:



$$S = A \oplus B \oplus C_{in}$$
$$C_{out} = AB + AC_{in} + BC_{in}$$



- Vários sinais agrupados para simplificar o circuito.
- Exemplo:



(a)



(b)



- Vários sinais agrupados para simplificar o circuito.

- Exemplo:

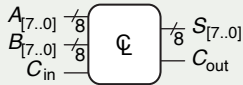


(a)

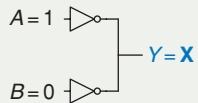


(b)

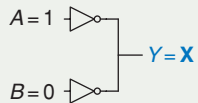
- Somador 8 bits:



- Valor **X** pode significar:
 - Circuito: Valor ilegal ou desconhecido



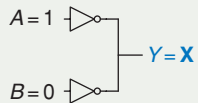
- Valor **X** pode significar:
 - Circuito: Valor ilegal ou desconhecido



- Simulador: Valor não inicializado



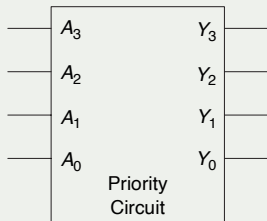
- Valor **X** pode significar:
 - Circuito: Valor ilegal ou desconhecido



- Simulador: Valor não inicializado
- Tabela Verdade: Não importa (*don't care*)



■ Tabela completa:

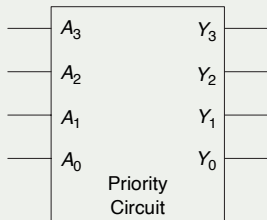


A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	0
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	0
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	0
1	0	1	0	1	0	0	0
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	0
1	1	1	0	1	0	0	0
1	1	1	1	1	0	0	0



Exemplo do uso do X

■ Tabela completa:



A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	0
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	0
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	0
1	0	1	0	1	0	0	0
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	0
1	1	1	0	1	0	0	0
1	1	1	1	1	0	0	0

■ Tabela Simplificada:

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	X	0	0	1	0
0	1	X	X	0	1	0	0
1	X	X	X	1	0	0	0



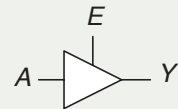
Além de 0's e 1's

- Valor **Z** significa que o nodo do circuito está flutuando ou alta impedância.
- Associado a um terceiro estado (*tristate*).



Além de 0's e 1's

- Valor **Z** significa que o nodo do circuito está flutuando ou alta impedância.
- Associado a um terceiro estado (*tristate*).
- Buffer *tristate*:

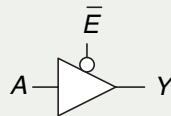


E	A	Y
0	0	Z
0	1	Z
1	0	0
1	1	1

Ativo alto

basicamente ele funciona
como uma chave, se enable
está ativado, ele não deixa o "a" passar

sempre que vemos a saída z, ele está em alta impedancia.



$\bar{E}$	A	Y
0	0	0
0	1	1
1	0	Z
1	1	Z

Ativo baixo



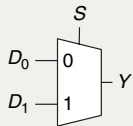
Blocos de Construção Combinacional

- Roteamento em circuitos.
- Implementação de lógica.



Multiplexador

- Roteamento em circuitos.
- Implementação de lógica.

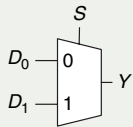


S	D_1	D_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



Multiplexador

- Roteamento em circuitos.
- Implementação de lógica.



S	D_1	D_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

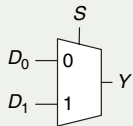


S	Y
0	D_0
1	D_1



Multiplexador

- Roteamento em circuitos.
- Implementação de lógica.

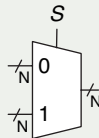


S	D ₁	D ₀	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

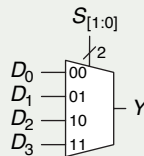


S	Y
0	D ₀
1	D ₁

- Barramentos :



- Multiplexador 4x1 :



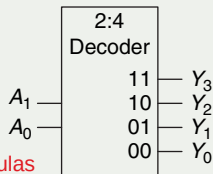
Dica:

Um multiplexador $N : 1$ precisar de $\log_2 N$ linhas de seleção.



Decodificador

- Decodifica um padrão para outro. Exemplos:
 - BCD $\rightarrow$ Display 7 segmentos;
 - Código de operação $\rightarrow$ sinais de controle;
 - Binário $\rightarrow$ *one hot*. N entradas para 2^N saídas.

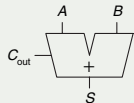


Esse é usado para habilitar todas as células de memória da linha 1, por exemplo. Vamos usar m_1

A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0



Half
Adder

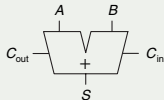


A	B	C _{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = A \oplus B$$

$$C_{out} = AB$$

Full
Adder



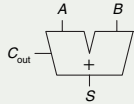
C _{in}	A	B	C _{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + AC_{in} + BC_{in}$$



Half
Adder

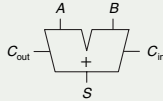


A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = A \oplus B$$

$$C_{out} = AB$$

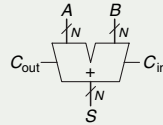
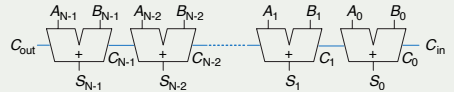
Full
Adder

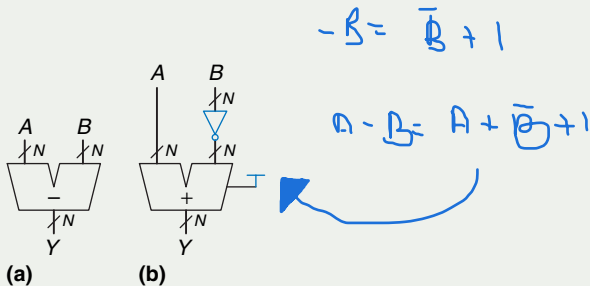


C_{in}	A	B	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + AC_{in} + BC_{in}$$



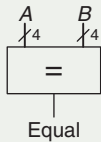


(a): Símbolo

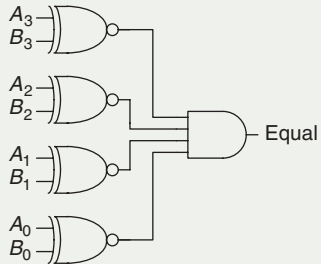
(b): Implementação ($Y = A + \bar{B} + 1 = A - B$)

■ Igualdade:

quando queremos
comparar, usamos
esse bloco de =

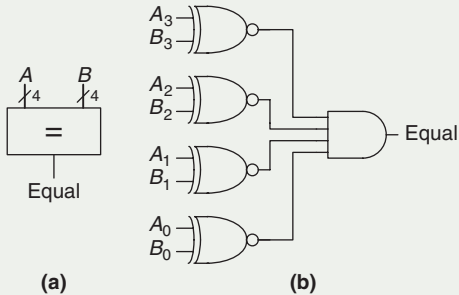


(a)

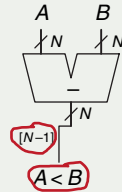


(b)

■ Igualdade:



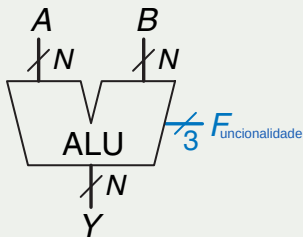
■ Menor que:



$$A - B > 0 \rightarrow A > B$$
$$A - B < 0 \rightarrow A < B$$

Pego o bit mais significativo da subtração,
se é zero o resultado é positivo e 1 é negativo

Unidade Lógica Aritmética - ULA



Todo processador tem essa unidade logica aritimetica, a arquitetura do pc é de 64 bits, significa que a unidade logica tem 64 bits, ta muito ligado a capacidade e rapidez dele

Para cada F eu defino uma funcionalidade diferente

$F_{2:0}$	Operação
000	A AND B
001	A OR B
010	A + B
011	not used
100	A AND $\bar{B}$
101	A OR $\bar{B}$
110	A - B
111	not used



Circuitos Sequenciais

- **Circuito combinacional:** saída depende exclusivamente de suas entradas atuais.
 - 1 Circuitos lógicos: portas, multiplexadores, de/codificadores de endereço etc.
 - 2 Circuitos aritméticos: somadores, subtratores, multiplicadores e divisores.



Lógica Combinacional versus Lógica Sequencial

■ **Circuito combinacional:** saída depende exclusivamente de suas entradas atuais.

1 Circuitos lógicos: portas, multiplexadores, de/codificadores de endereço etc.

2 Circuitos aritméticos: somadores, subtratores, multiplicadores e divisores.

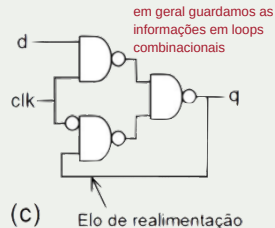
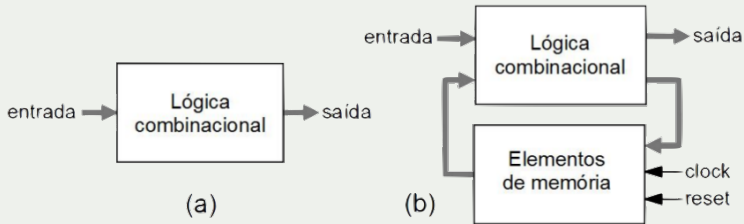
= saída depende só da entrada

teremos portas, decodificadores de endereços...

■ **Circuito sequencial:** o valor das saídas pode depender das entradas e de valores anteriores das saídas.

= Dependendo do estado que o circuito está, ele depende da saída.

Ex contador, então a saída está vinculada a o estado interno do nosso circuito



teremos shefte reter, contadores...



- **Estado:** É uma combinação dos valores das saídas do circuito.
- **Transição:** Quando as entradas do circuito mudam o seu valor, os valores das saídas podem mudar em consequência disso, dizendo-se então que houve uma *transição* de estado.
- **Clock:** É um sinal periódico extremamente importante em circuitos sequenciais, pois permite condicionar o ritmo em que as operações evoluem.

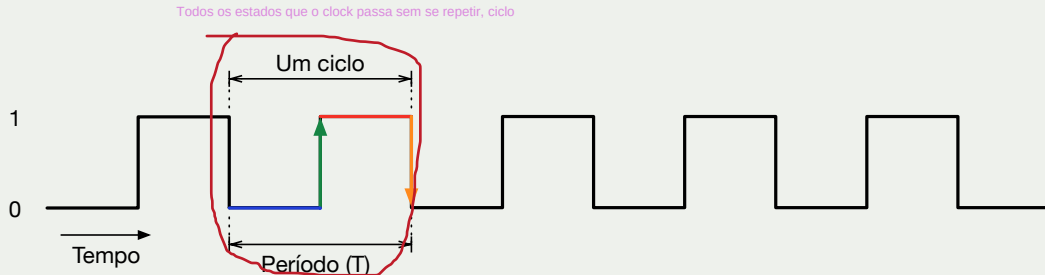
São os estados por onde
nosso circuito vai passar.
Um contador de 4 bits tem 2
na 4 estados, ou seja 16;

O termo “sequencial” deriva do fato do estado para o qual um circuito transita poder depender não apenas da variação ocorrida nas entradas como também do estado anterior e até mesmo da sequência de estados, ou seja, do histórico dos estados, pela qual o circuito transitou.



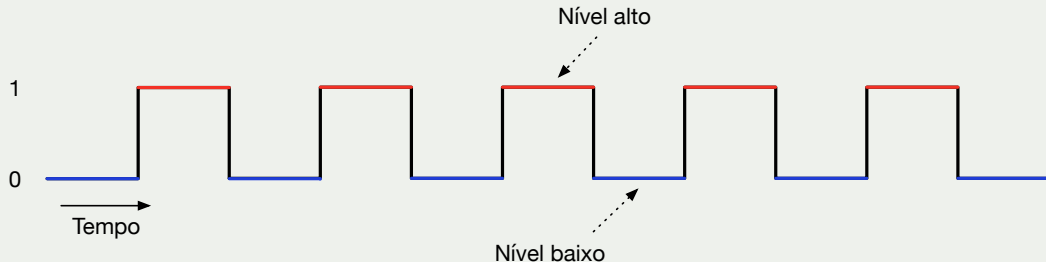
Clock – Conceitos básicos

- **Um ciclo de clock:** É uma oscilação única de todos os valores possíveis do sinal.
- **Período:** Intervalo de tempo de um ciclo de *clock*, medido em unidades de tempo.
- **Frequência:** Quantidade de ciclos por segundo, medido em Hertz (Hz).
 - O período é o inverso da frequência, ou seja, $F = \frac{1}{T}$. Quanto menor nosso período, mais rápido para efetuar a ação



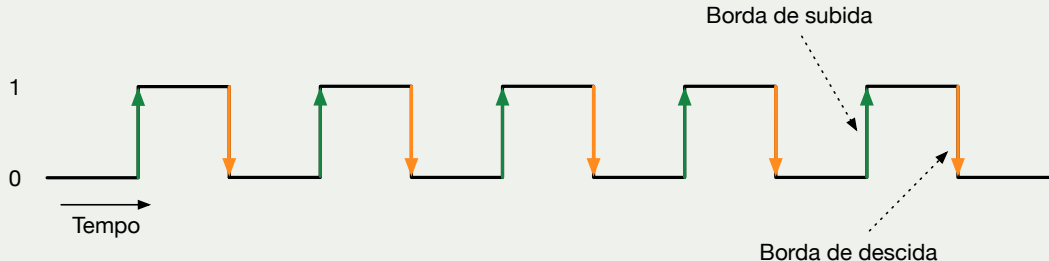
Clock – Nível 0 ou 1

- **Nível:** É o tempo em que o sinal está em alto (1) ou baixo (0)
- Alguns blocos digitais são sensíveis a nível, ou seja, reagem ao nível baixo ou alto.



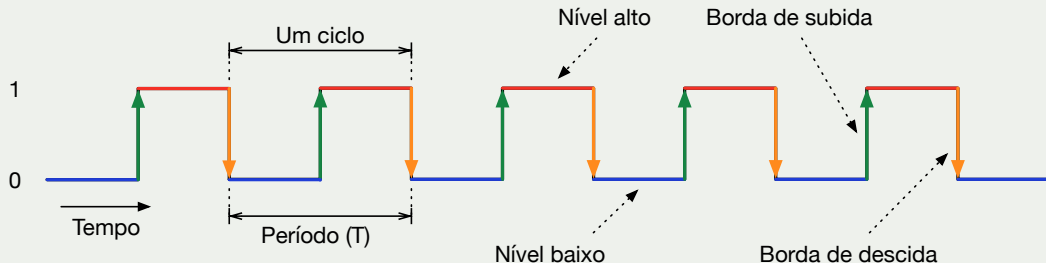
Clock – Borda de subida ou descida

- **Borda de subida:** Transição do nível baixo para o alto.
- **Borda de descida:** Transição do nível alto para o baixo.
- Forma de sincronização mais fina dos circuitos digitais. Alguns blocos são sensíveis à borda, de subida ou descida.

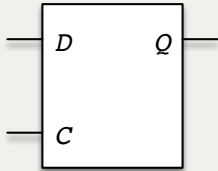


Clock – Resumo

Na prática, o *clock* é um sinal síncrono que flutua entre dois níveis de tensão em um intervalo fixo, formando, idealmente, uma onda quadrada que é utilizada como entrada para sincronizar todas as ações dos componentes internos de um sistema. Os níveis de tensão podem ser abstraídos para o valor alto (nível 1) e o valor baixo (nível 0).



■ Latch D:



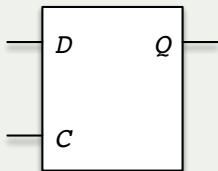
C	Q
0	Q_{prev}
1	D

■ Funcionamento:

- Quando a entrada C está em 1, o *latch* fica transparente, ou seja, deixa passar a informação que está na entrada D para a saída Q .
- Quando a entrada C está em 0, o *latch* tranca a passagem e mantém memorizado o valor de Q .



■ Latch D:

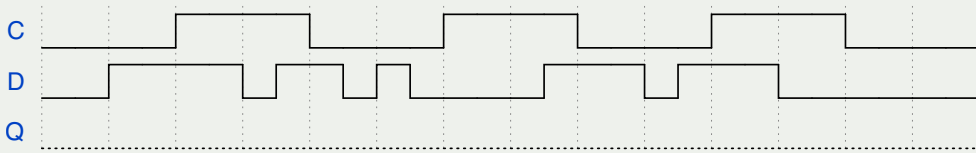


C	Q
0	Q_{prev}
1	D

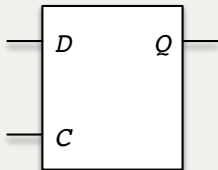
■ Funcionamento:

- Quando a entrada C está em 1, o *latch* fica transparente, ou seja, deixa passar a informação que está na entrada D para a saída Q .
- Quando a entrada C está em 0, o *latch* tranca a passagem e mantém memorizado o valor de Q .

■ Sensível a nível alto:



■ Latch D:

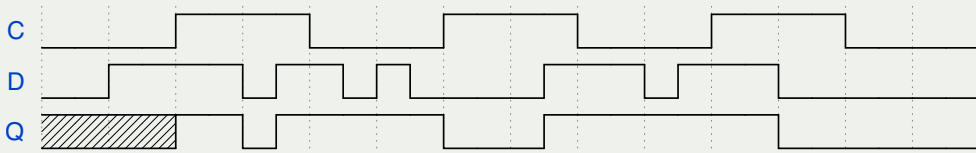


C	Q
0	Q_{prev}
1	D

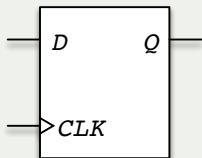
■ Funcionamento:

- Quando a entrada *C* está em 1, o *latch* fica transparente, ou seja, deixa passar a informação que está na entrada *D* para a saída *Q*.
- Quando a entrada *C* está em 0, o *latch* tranca a passagem e mantém memorizado o valor de *Q*.

■ Sensível a nível alto:



■ Flip-flop D:



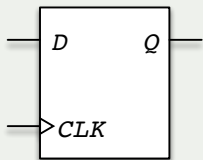
CLK	Q
0	Q_{prev}
1	Q_{prev}
	D

■ Funcionamento:

- Somente na borda de subida do *clock* (CLK), o *flip-flop* permite a passagem de informação da entrada D para a saída Q .
- Em todos os outros momentos do ciclo de *clock*, o *flip-flop* mantém o valor de Q .



■ Flip-flop D:

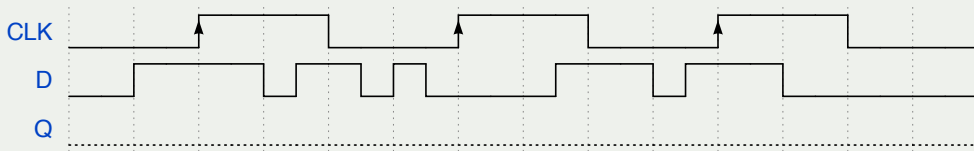


CLK	Q
0	Q_{prev}
1	Q_{prev}
	D

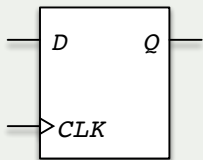
■ Funcionamento:

- Somente na borda de subida do *clock* (CLK), o *flip-flop* permite a passagem de informação da entrada D para a saída Q .
- Em todos os outros momentos do ciclo de *clock*, o *flip-flop* mantém o valor de Q .

■ Sensível à borda de subida:



■ Flip-flop D:

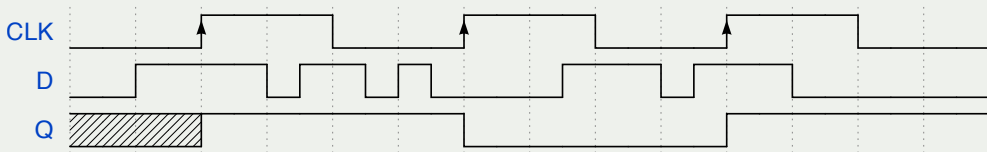


CLK	Q
0	Q_{prev}
1	Q_{prev}
	D

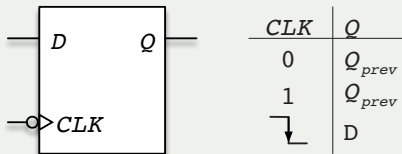
■ Funcionamento:

- Somente na borda de subida do *clock* (CLK), o *flip-flop* permite a passagem de informação da entrada *D* para a saída *Q*.
- Em todos os outros momentos do ciclo de *clock*, o *flip-flop* mantém o valor de *Q*.

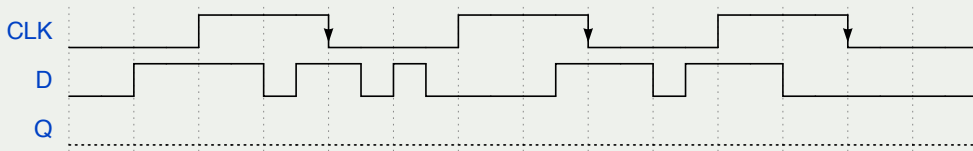
■ Sensível à borda de subida:



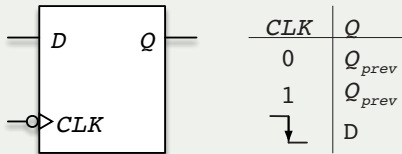
- É possível criar um *flip-flop* sensível a borda de descida:



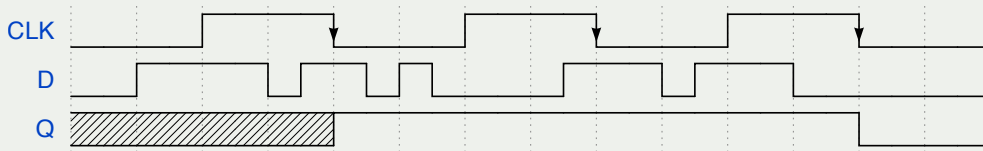
- Funcionamento:



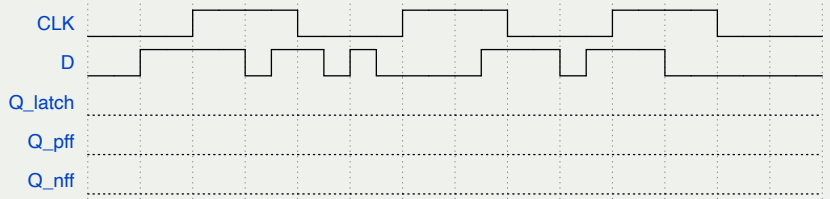
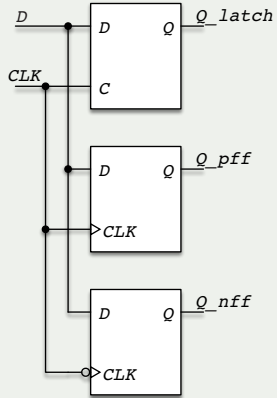
- É possível criar um *flip-flop* sensível a borda de descida:



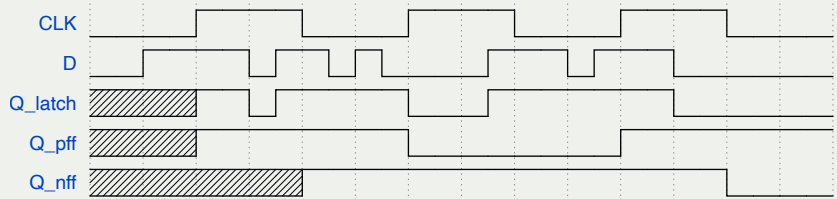
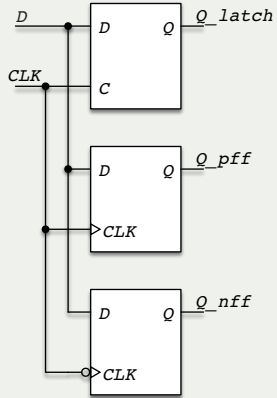
- Funcionamento:



Flip-Flop vs. Latch



Flip-Flop vs. Latch



Flip-flop – Opcionais

- É possível adicionar mais alguns sinais de controle no *flip-flop*:

- **Reset assíncrono:** zera a saída Q independente do *clock*. Utilizado na inicialização do sistema.

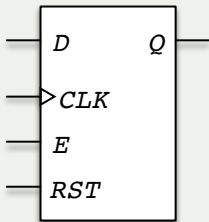
Retesa antes mesmo do clock está estável, ou seja, no começo, na saída já sai em zero

- **Enable:** Habilita ou desabilita a escrita no *flip-flop*. Pode ser ativo em alto ou baixo.

Se o reset tá em zero, aí sim o enable ativa. Se estiver em zero, não importa o clock, o q prev fica em zero

- Exemplo: *Flip-flop* sensível à borda de subida com *enable* ativo alto e *reset* assíncrono.

Para que o sinal de d passe para q , o rst tem que estar em zero, o $e = 1$ e o clock precisa tá em subida, se não ele vai manter o valor anterior.



RST	E	CLK	Q
1	X	X	0
0	0	X	Q_{prev}
0	1	0	Q_{prev}
0	1	1	Q_{prev}
0	1		D



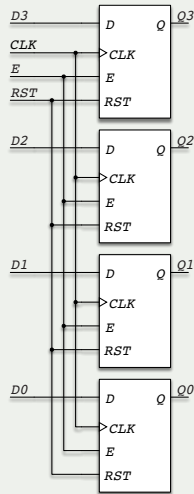
- Um registrador é um conjunto de elementos de memória.
- Um registrador de N bits é um banco de N *flip-flops* que compartilham uma mesma entrada de *clock*. Assim, todos os bits do registrador são atualizados ao mesmo tempo.
- Registrador é o bloco de construção chave para a maioria dos circuitos sequenciais.



Registrador

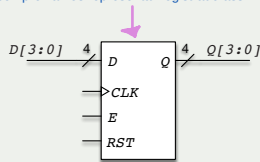
- Um registrador é um conjunto de elementos de memória.
- Um registrador de N bits é um banco de N *flip-flops* que compartilham uma mesma entrada de *clock*. Assim, todos os bits do registrador são atualizados ao mesmo tempo.
- Registrador é o bloco de construção chave para a maioria dos circuitos sequenciais.

■ Registrador de 4 bits:



(a) Circuito

sempre vamos representar registrado assim



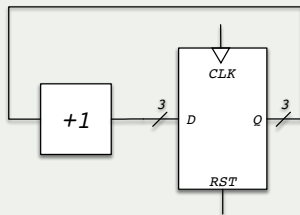
(b) Símbolo



- Contador binário de 3 bits:



■ Contador binário de 3 bits:

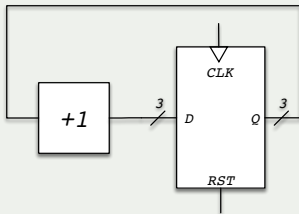


Ele vai fazer a contagem até o limite da representação, que é 2 na 3 -1, que é 7;

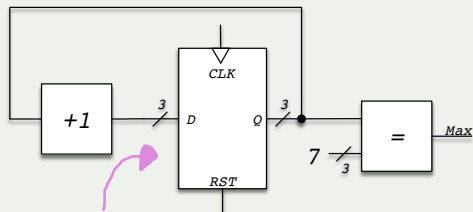


Contador

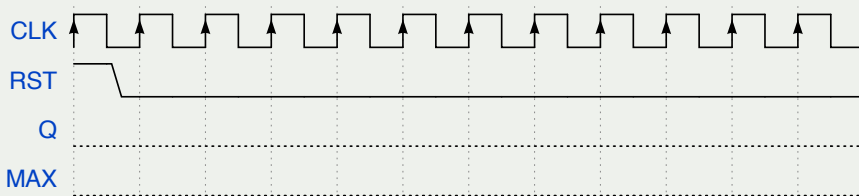
■ Contador binário de 3 bits:



■ Com pulso de máximo:

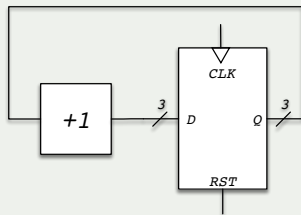


Na hora que a contagem for = 7 dá o pulso para mim
durante todo resto da contagem o pulso é zero

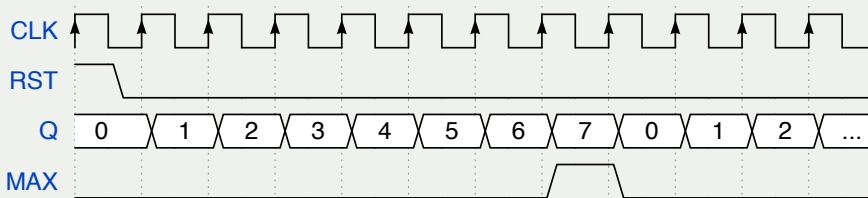
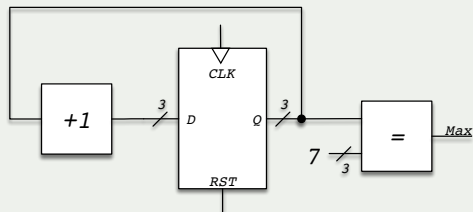


Contador

■ Contador binário de 3 bits:



■ Com pulso de máximo:



FIM!